

Practical 2:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

#define BUFFER_SIZE 4096

int main(int argc, char *argv[]) {
    int src_fd, dest_fd;
    ssize_t bytes_read, bytes_written;
    char buffer[BUFFER_SIZE];

    if (argc != 3) {
        fprintf(stderr, "Usage: %s <source_file> <destination_file>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    // 1. Open source file for reading only
    src_fd = open(argv[1], O_RDONLY);
    if (src_fd < 0) {
        perror("Error opening source file");
        exit(EXIT_FAILURE);
    }

    // 2. Open/Create destination file for writing (Read/Write for owner)
    dest_fd = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dest_fd < 0) {
        perror("Error opening/creating destination file");
        close(src_fd);
        exit(EXIT_FAILURE);
    }

    // 3. Read from source and write to destination in chunks
    while ((bytes_read = read(src_fd, buffer, BUFFER_SIZE)) > 0) {
        bytes_written = write(dest_fd, buffer, bytes_read);
        if (bytes_written != bytes_read) {
            perror("Write error occurred");
        }
    }
}
```

[Read 5/1 lin

```

src_fd = open(argv[1], O_RDONLY);
if (src_fd < 0) {
    perror("Error opening source file");
    exit(EXIT_FAILURE);
}

// 2. Open/Create destination file for writing (Read/Write for owner)
dest_fd = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (dest_fd < 0) {
    perror("Error opening/creating destination file");
    close(src_fd);
    exit(EXIT_FAILURE);
}

// 3. Read from source and write to destination in chunks
while ((bytes_read = read(src_fd, buffer, BUFFER_SIZE)) > 0) {
    bytes_written = write(dest_fd, buffer, bytes_read);
    if (bytes_written != bytes_read) {
        perror("Write error occurred");
        close(src_fd);
        close(dest_fd);
        exit(EXIT_FAILURE);
    }
}

if (bytes_read < 0) {
    perror("Read error occurred");
}

// 4. Close both file descriptors
close(src_fd);
close(dest_fd);

printf("File copied successfully from '%s' to '%s'.\n", argv[1], argv[2]);
return 0;

```

[illegible]